**SBOM generation results**

In this lab, I created a Software Bill of Materials for the NG911 repository using two different tools: Syft and Trivy. Both tools analyze a codebase and generate a list of the components, dependencies, and libraries used in the software. When I used Syft, it generated an SBOM in SPDX JSON format, while Trivy produced one in CycloneDX format. In both cases, the tools were able to identify several components in the repository. One thing I noticed is that the two SBOMs look a bit different in terms of format and structure. The SPDX SBOM from Syft focuses more on detailed package information and relationships between components. On the other hand, the CycloneDX SBOM from Trivy seems more structured for use with security tools and puts more emphasis on vulnerability tracking. I also saw slight differences in the number of components listed and how dependencies were represented. Overall, even though both tools are doing the same basic job, they seem to be designed with slightly different purposes.

**Vulnerability analysis (top 5 CVEs or rationale for zero matches)**

After generating the SBOM, I used Grype to perform a vulnerability analysis. Grype scans the SBOM and identifies known vulnerabilities associated with the components.

Five vulnerabilities identified from the analysis:

| NAME | INSTALLED | FIXED IN | TYPE | VULNERABILITY | SEVERITY | EPSS | RISK |
|---|---|---|---|---|---|---|---|
| cryptography | 46.0.5 | 46.0.7 | python | GHSA-p423-j2cm-9vmq | Medium | < 0.1% (5th) | < 0.1 |
| pytest | 8.3.3 | 9.0.3 | python | GHSA-6w46-j5rx-g56g | Medium | < 0.1% (0th) | < 0.1 |
| cryptography | 46.0.5 | 46.0.6 | python | GHSA-m959-cc7f-wv43 | Low | < 0.1% (0th) | < 0.1 |
| requests | 2.32.4 | 2.33.0 | python | GHSA-gc5v-m9x4-r6x2 | Medium | < 0.1% (0th) | < 0.1 |
| lxml | 5.2.2 | 6.1.0 | python | GHSA-vfmq-68hx-4jfw | High | N/A | N/A |

One vulnerability that stood out was the lxml high severity issue, which affects how XML data is processed. If exploited, this type of vulnerability could allow attackers to manipulate or inject malicious data, potentially leading to data leaks or system compromise. This analysis shows how important it is to continuously monitor dependencies, since even commonly used libraries can contain vulnerabilities.

**Screenshots**

```
Command executed 10 mins ago and took 0ms
ady exists and is not an empty directory.
● @elina0619 → /workspaces/eng298-sp26-mod7-sbom-lab1 (main) $ cd ng911-dev
● @elina0619 → /workspaces/eng298-sp26-mod7-sbom-lab1/ng911-dev (main) $ syft . -o spdx-json > ../deliv
erables/sbom_syft_spdx.json
 ✔ Indexed file system                                                                          .
 ✔ Cataloged contents                   cdb4ee2aea69cc6a83331bbe96dc2caa9a299d21329efb0336fc02a82e1839a8
   ├── ✔ Packages                        [129 packages]
   ├── ✔ Executables                     [0 executables]
   ├── ✔ File metadata                   [10 locations]
   └── ✔ File digests                    [10 files]  [0000]  WARN no explicit name and version provid
● @elina0619 → /workspaces/eng298-sp26-mod7-sbom-lab1/ng911-dev (main) $ trivy fs . --format cyclonedx
--output ../deliverables/sbom_trivy_cdx.json
2026-04-24T15:41:23Z    INFO    "--format cyclonedx" disables security scanning. Specify "--scanners
vuln" explicitly if you want to include vulnerabilities in the "cyclonedx" report.
2026-04-24T15:41:23Z    INFO    [python] Licenses acquired from one or more METADATA files may be sub
ject to additional terms. Use `--debug` flag to see all affected packages.
2026-04-24T15:41:23Z    INFO    [npm] To collect the license information of packages, "npm install" n
eeds to be performed beforehand dir="test_suite/test_files/_old/TPLan_Config/VS_Code/node_modules"
2026-04-24T15:41:23Z    INFO    Number of language-specific files       num=2
● @elina0619 → /workspaces/eng298-sp26-mod7-sbom-lab1/ng911-dev (main) $ ls ../deliverables/
README.md  sbom_syft_spdx.json  sbom_trivy_cdx.json
```

```
● @elina0619 → /workspaces/eng298-sp26-mod7-sbom-lab1/ng911-dev (main) $ grype sbom:../deliverables/sbo
m_syft_spdx.json -o table > ../deliverables/vuln_analysis_grype.txt
 ✔ Vulnerability DB                [updated]
 ✔ Scanned for vulnerabilities     [5 vulnerability matches]
   ├── by severity: 0 critical, 1 high, 3 medium, 1 low, 0 negligible
   └── by status:   5 fixed, 0 not-fixed, 0 ignored
● @elina0619 → /workspaces/eng298-sp26-mod7-sbom-lab1/ng911-dev (main) $ head -20 ../deliverables/vuln_
analysis_grype.txt
NAME          INSTALLED  FIXED IN  TYPE    VULNERABILITY        SEVERITY  EPSS          RISK
cryptography  46.0.5     46.0.7    python  GHSA-p423-j2cm-9vmq  Medium    < 0.1% (5th)  < 0.1
pytest        8.3.3      9.0.3     python  GHSA-6w46-j5rx-g56g  Medium    < 0.1% (0th)  < 0.1
cryptography  46.0.5     46.0.6    python  GHSA-m959-cc7f-wv43  Low       < 0.1% (0th)  < 0.1
requests      2.32.4     2.33.0    python  GHSA-gc5v-m9x4-r6x2  Medium    < 0.1% (0th)  < 0.1
lxml          5.2.2      6.1.0     python  GHSA-vfmq-68hx-4jfw  High      N/A           N/A
● @elina0619 → /workspaces/eng298-sp26-mod7-sbom-lab1/ng911-dev (main) $ ls ../deliverables/
README.md  sbom_syft_spdx.json  sbom_trivy_cdx.json  vuln_analysis_grype.txt
● @elina0619 → /workspaces/eng298-sp26-mod7-sbom-lab1/ng911-dev (main) $ cd ..
```

```
● @elina0619 → /workspaces/eng298-sp26-mod7-sbom-lab1 (main) $ git commit -m "finish sbom lab"
You don't have write access to the obrien-casey/eng298-sp26-mod7-sbom-lab1 repository, so you cannot
push changes to it.
To obtain write access we will point this codespace at your fork of obrien-casey/eng298-sp26-mod7-sbo
m-lab1, creating that fork if it doesn't exist.

Would you like to proceed? y

From https://github.com/elina0619/eng298-sp26-mod7-sbom-lab1
 * branch            main       -> FETCH_HEAD
 * [new branch]      main       -> origin/main
branch 'main' set up to track 'origin/main'.

We've configured write access for you! Use the remote 'origin' to interact with your fork. Use the re
mote 'upstream' to interact with the parent repository.
[main 8a2919c] finish sbom lab
 3 files changed, 1954 insertions(+)
 create mode 100644 deliverables/sbom_syft_spdx.json
 create mode 100644 deliverables/sbom_trivy_cdx.json
 create mode 100644 deliverables/vuln_analysis_grype.txt
○ @elina0619 → /workspaces/eng298-sp26-mod7-sbom-lab1 (main) $
● @elina0619 → /workspaces/eng298-sp26-mod7-sbom-lab1 (main) $ git push
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 2 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 20.86 KiB | 3.48 MiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/elina0619/eng298-sp26-mod7-sbom-lab1.git
   a7b4d65..8a2919c  main -> main
○ @elina0619 → /workspaces/eng298-sp26-mod7-sbom-lab1 (main) $
```

### Reflection on insights and process

This lab really helped me understand just how complex modern software systems are and how heavily they depend on external libraries and components. Before doing this, I didn't fully realize how important it is to keep track of every single part of a system. Working with SBOM tools like Syft and Trivy made it clear that transparency is a big part of security. SBOMs support important security principles like defense in depth and complete mediation because they make it possible to see and verify all components, instead of leaving parts of the system unchecked. The vulnerability analysis using Grype stood out to me the most. It was surprising to see that even widely used libraries can have real security vulnerabilities. This really shows why it's important to regularly scan systems and keep dependencies up to date. Overall, this lab gave me a better understanding of software supply chain security. This connects directly to my interest in cybersecurity, and I can definitely see how these tools are used in real-world situations.